

Machine Learning (MLOps) AI-Based Football Analytics Using Computer Vision and MLOps

Team 3

USN	Name
1RV23AI072	Himashree P
1RV23AI074	Preetam Baheti
1RV23AI076	Priyansh A P
1RV23AI077	R Daksharani

Agenda

- Introduction
- Motivation (Why this title)
- Problem Definition
- Objectives
- Data set Description
- Risk Analysis
- Tools and Techniques used
- Phase I : Machine Learning Model and its analysis
- Phase II : Building MLOPs

INTRODUCTION

AI-Based Football Analytics Using Computer Vision and MLOps

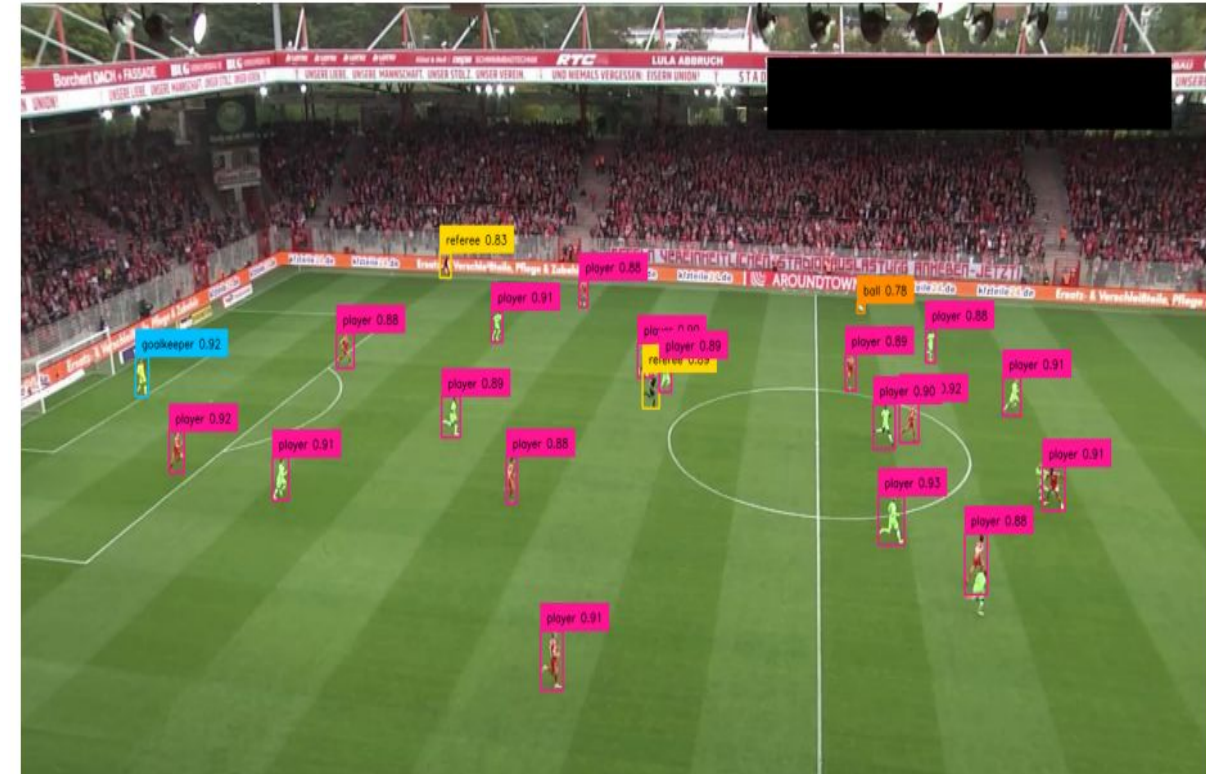
An End-to-End Machine Learning Pipeline for Automated Sports Analysis.

- Football analysis traditionally depends on manual video review and expert observation.
- With the rise of Artificial Intelligence and Computer Vision, it is now possible to automatically analyze player movements and match dynamics from video data.
- Broadcast football videos are widely available but are unstructured and difficult to analyze at scale.
- Machine Learning enables detection and tracking of players and the ball, while MLOps ensures that these models are reliable, scalable, and maintainable.

This project presents an end-to-end AI-driven football analytics system that integrates ML models with MLOps practices for real-world usability.

Football Is Rich in Data and Manual Analysis Is Limited.

Traditional analysis is slow, subjective, and difficult to scale.



Computer vision enables automatic detection and tracking of players and the ball.

MLOps ensures models are versioned, deployed, monitored, and continuously improved.

Motivation(Why this title?)

- Football analytics is rapidly adopting AI-driven decision-making, but most solutions are expensive and proprietary.
- Broadcast match videos are easily available but underutilized due to lack of automated analysis tools.
- The title emphasizes AI-based automation, highlighting the use of machine learning and computer vision techniques rather than manual analysis.
- The term Football Analytics reflects the project's focus on extracting tactical and spatial insights such as player positioning, movement patterns, and ball trajectories.
- Computer Vision is explicitly included to indicate that visual data from broadcast match footage is the primary input to the system.
- The inclusion of MLOps in the title signifies that the project goes beyond model development to cover deployment, versioning, monitoring, and lifecycle management.
- Overall, the title represents an end-to-end, production-oriented machine learning system aligned with real-world applications and the MLOps syllabus.

Problem Statement

The increasing availability of broadcast football videos presents an opportunity to perform automated match analysis using machine learning techniques. However, extracting meaningful tactical information such as player positions, ball movement, and team formations from raw video data is a complex task due to factors like occlusion, camera motion, and varying viewpoints. Additionally, most machine learning solutions focus only on model development and lack systematic approaches for data management, deployment, monitoring, and version control. Therefore, there is a need to develop an **end-to-end AI-based football analytics system** that integrates computer vision models with **MLOps practices** to ensure reliable performance, scalability, and maintainability throughout the machine learning lifecycle.

Objectives

Model Development

To develop computer vision models for accurate detection and tracking of players, referees, and the ball from broadcast football videos.

Spatial & Tactical Analysis:

To map detected entities onto a standardized pitch and generate analytical insights such as player movement, ball trajectory, and team control regions.

Model Evaluation:

To evaluate the performance of the machine learning models using appropriate accuracy, tracking, and efficiency metrics.

MLOps Implementation:

To implement MLOps practices including data versioning, experiment tracking, deployment, and monitoring to ensure scalability and maintainability.

Risk Analysis

Risks identified:

Risk ID	Description
R1	Dataset inconsistency or low-quality annotations
R2	Model accuracy drop due to overfitting
R3	Server downtime or cloud resource unavailability
R4	Total system failure during integration or demo
R5	High computational load / GPU bottleneck

dataset

🔍 Search

Sign In

Register



DEUTSCHE FUSSBALL LIGA E.V. · FEATURED CODE COMPETITION · 3 YEARS AGO

Late Submission

...

DFL - Bundesliga Data Shootout

Identify plays based upon video footage



Overview

Data

Code

Models

Discussion

Leaderboard

Rules

Dataset Description

NOTE: DATA IS NO LONGER AVAILABLE FOR THIS COMPLETED COMPETITION.

The competition dataset comprises video recordings of nine football games divided into halves. You're challenged to detect three kinds of player events, both the time of occurrence and the type, within these videos. See the [Event Descriptions](#) page for a full description of each event type.

Files

1 files

Size

0 B

Type

+ 1 other

Dataset Description

Dataset Description (DFL Bundesliga Data Shootout – Kaggle)

- Dataset contains short football match video clips from Bundesliga games
- Clips show real broadcast footage with players, the ball, referees, and match context
- Data is intended for event recognition and football analytics research
- Videos include different camera angles, lighting, occlusions, and motion
- Often used for object detection, tracking, and tactical modelling tasks
- Frames can be extracted and annotated for training deep learning models

Risk Analysis

Risk Likelihood vs Consequence Matrix

Likelihood / Consequence	Insignificant	Minor	Moderate	Major	Catastrophic
Most Likely to Occur		R5 – High computational load			
Likely to Occur Frequently			R1 – API/pipeline failure		
Possibly Occurs Sometimes		R3 – Server downtime	R2 – Model accuracy drop		
Unlikely to Occur			R7 – Integration error		
Rarely Occurs		R6 – Data storage failure		R4 – System failure during testing	

Why We Chose This Model?

What model did we choose?

YOLOv8 (You Only Look Once) Object Detection model.

Why we chose it?

- **Fast detection** → gives results in real-time / near real-time
- **Good accuracy** → strong performance for small objects like players
- **Easy training** → works directly with YOLO-format datasets (Roboflow output)
- **Pre-trained weights available** → starts with a strong base, trains faster
- **Production-friendly** → easy to export and deploy later

Comparison of models

Metric	YOLOv8-m	YOLOv9-m	YOLOv11-m
mAP@0.5	83.6%	88.9%	87.4%
mAP@0.5:0.95	63.2%	69.8%	68.1%
Precision	84.5%	89.7%	88.2%
GPU Memory Usage	4.2 GB	5.6 GB	4.8 GB
Avg GPU Utilization	64%	78%	69%
Training Time / epoch	6.8	7.9	7.1

Tools and Techniques used

Tool	Purpose
Weights & Biases (W&B)	Experiment tracking, model performance monitoring.
Roboflow	Dataset management, annotation, and model training interface.
MLflow	Model versioning and reproducibility.
Docker	Containerization for consistent deployment.
Google Colab / Kaggle Notebooks	Cloud-based model training and quick experimentation.

DVC: Data Management

What is it?

DVC helps us save and track dataset versions (like Git, but for data).

How it helps?

- Our dataset is inside the `data/` folder
- DVC tracks that dataset version
- If dataset changes, DVC saves a new version
- Any teammate can run `dvc pull` and get the same dataset

Football Players Detection Dataset

[Analytics Report](#)

□ DATASET STATISTICS

Total Images: 663
Total Labels: 663
Total Size: 144.86 MB
Number of Classes: 4
Classes: ball, goalkeeper, player, referee

□ DATA SPLITS

TRAIN | Images: 612 | Labels: 612 | Size: 133.75 MB
VALID | Images: 38 | Labels: 38 | Size: 8.23 MB
TEST | Images: 13 | Labels: 13 | Size: 2.89 MB

□ DVC VERSION CONTROL

Hash (MD5): 55a0d48967098495b3c63dce542b365e...
Tracked Size: 144.87 MB
Number of Files: 1,329
DVC File: data/football-players-detection.dvc

□ DATASET SOURCE

Project: football-players-detection-3zvbc
Workspace: roboflow-jvuqo
Version: 1
License: CC BY 4.0

SUMMARY

□ Dataset successfully versioned with DVC
□ 663 images across 3 splits
□ 4 classes: ball, goalkeeper, player, referee
□ Total dataset size: 144.86 MB

Roboflow: Experiment Tracking

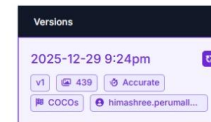
What is it?

Roboflow is a platform to store, manage, and download labeled football datasets.

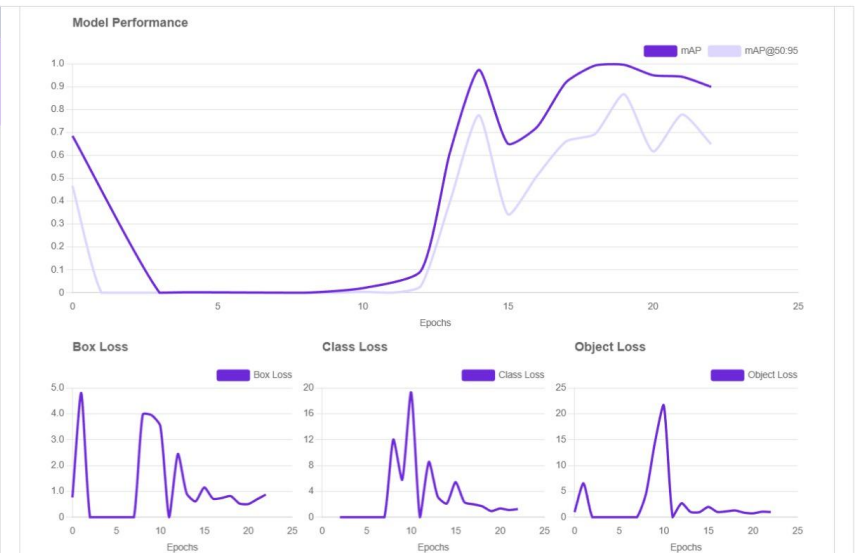
How it helps?

- We download the dataset in YOLOv8 format
- It automatically gives:
 - `train/` folder
 - `valid/` folder
 - `data.yaml` file
- Roboflow also supports dataset versions (v1, v2...)

Versions



+ Create New Version



Weights & Biases (W&B): Training & Tracking

What is it?

W&B is a tool to record training results like accuracy and loss.

How it helps?

- It stores for each training:
 - mAP / accuracy metrics
 - loss graphs
 - epochs, batch size, learning rate
- We can compare all runs in one place



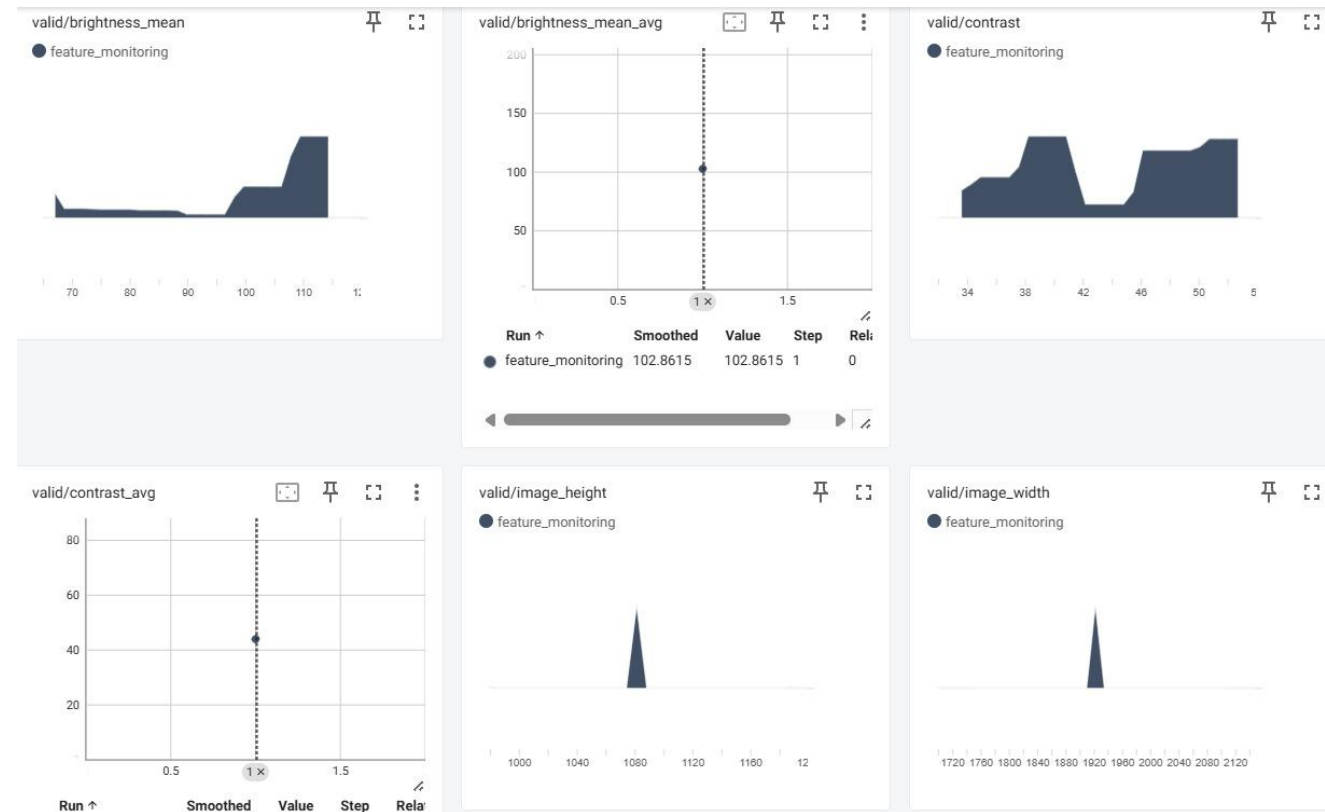
TensorBoard: Monitoring & Governance

What is it?

TensorBoard shows training graphs like loss vs epochs and accuracy vs epochs.

How it helps?

- We can see:
 - training loss decreasing or not
 - validation performance
 - overfitting (train good but valid bad)



Phase II – MLOPs Life Cycle

	Component	MLOPs Tools
1	Data Management	<u>DVC</u>
2	Experiment Tracking and Version Control	<u>GitHub</u>
3	Automation and Pipeline Orchestration	<u>Docker, Prefect</u>
4	Model Deployment & Serving	<u>Roboflow,Weights and Biases</u>
5	Monitoring & Governance and collaboration	<u>Tensorboard</u>

Phase II – MLOPs Life Cycle

Experiment Tracking and Model Versioning (MLflow)

- MLflow is used to log each experiment run
- We store model parameters, accuracy metrics, and run details
- Each trained model version is saved and can be promoted or rolled back
- This allows consistent comparison across experiments
- The tracking UI gives a clear view of which model performs best
- Model Registry enables structured version management

Phase II – MLOPs Life Cycle

Data and Model Quality Monitoring (Evidently AI)

- Evidently AI monitors dataset and model behaviour over time
- It checks for data drift, class imbalance changes, and performance drops
- Reference and current datasets are compared continuously
- This helps detect when retraining is required
- Reports are structured and repeatable, suitable for audit and review
- The goal is to maintain stable model performance in changing environments

Phase II – MLOPs Life Cycle

System and GPU Usage Monitoring (Weights & Biases)

- Weights & Biases tracks system resource usage during training and inference
- GPU utilization, GPU memory use, CPU usage, and RAM are monitored in real time
- This helps identify whether the model is compute-efficient
- Performance bottlenecks or crashes can be traced back to resource limits
- Logs remain stored and can be revisited later for analysis
- Together with MLflow, W&B adds the operational visibility layer

Phase II – MLOPs Life Cycle

Workflow Automation and Reproducibility (Prefect / GitHub Actions)

- Prefect or GitHub Actions automate the pipeline steps
- Training, evaluation, logging, and monitoring run in a defined sequence
- This removes manual intervention and improves reliability
- Jobs are repeatable, scheduled, and traceable
- Failures can be identified at the workflow level
- Automation ensures that MLOps practices are consistently applied

Phase II – MLOPs Life Cycle

[PRefect.ai](https://docs.prefect.io/v3/concepts/server#how-to-guides)

```
See https://docs.prefect.io/v3/concepts/server#how-to-guides for more information on running a dedicated Prefect server.
00:21:38.793 | INFO      | Flow run 'accomplished-llama' - Beginning flow run 'accomplished-llama' for flow 'Football Tracking MLOps Pipeline'
  ♦ Extracting features...
  🗂 Processing train images...
  ❌ Folder not found: .\train
  ⚠ No images found in train
  🗂 Processing valid images...
  ❌ Folder not found: .\valid
  ⚠ No images found in valid
  🗂 Processing test images...
  ❌ Folder not found: .\test
  ⚠ No images found in test
00:21:41.302 | INFO      | Task run 'extract_features-48f' - Finished in state Completed()
  ♦ Running model inference...
C:\Users\hperu\AppData\Local\Programs\Python\Python312\Lib\site-packages\torch\cuda\__init__.py:61: FutureWarning: The pynvml package is deprecated. Please install nvidia-ml-py instead. If you did not install pynvml directly, please report this to the maintainers of the package that installed pynvml for you.
  import pynvml # type: ignore[import]
wandb: Currently logged in as: himashreep-ai23 (chandanamn-cs23-na) to https://api.wandb.ai. Use 'wandb login --relogin' to force relogin
wandb: Tracking run with wandb version 0.23.1
wandb: Run data is saved locally in C:\Users\hperu\football2\wandb\run-20251230_002200-egfc4t8c
wandb: Run 'wandb offline' to turn off syncing.
wandb: Syncing run yolo-inference
wandb: View project at https://wandb.ai/chandanamn-cs23-na/football-mlops
wandb: View run at https://wandb.ai/chandanamn-cs23-na/football-mlops/runs/egfc4t8c
```

Phase II – MLOPs Life Cycle

Environment Standardization and Deployment Readiness (Docker)

- Docker packages the code, dependencies, and runtime into a container
- The same image runs identically on any compatible machine
- This eliminates “works on my system” issues
- Makes scaling, collaboration, and deployment simpler
- Supports GPU-based execution when combined with NVIDIA runtime
- Forms the base layer supporting all other MLOps tools